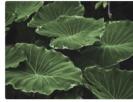


WE LIKE TO
SHOP



Pet Experience Days
★★★★☆ \$27.76

[View details](#)



Grow Your Own Spy Kit
★★★★☆ \$94.08

[View details](#)



First Impression Costumes
★★★★☆ \$11.60

[View details](#)



Sprout More Brain Power
★★★★☆ \$73.28

[View details](#)



High-End Gift Wrapping
★★★★☆ \$50.58

[View details](#)



Fur Babies
★★★★☆ \$89.36

[View details](#)



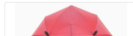
Robot Home Security Buddy
★★★★☆ \$49.53

[View details](#)



Portable Hat
★★★★☆ \$61.16

[View details](#)



Actividad 11

File path traversal, validation of start of path

Guerrero Hurtado Lizbeth

Marco Teórico

¿Qué es File Path Traversal?

Es una vulnerabilidad de control de acceso que ocurre cuando una aplicación web no sanitiza ni valida adecuadamente las entradas proporcionadas por el usuario al construir rutas del sistema de archivos. Permite a un atacante escapar del directorio web público (webroot) mediante secuencias de navegación y acceder a archivos sensibles del servidor subyacente.

Validation of Start of Path:

En este escenario, el desarrollador implementa un mecanismo de defensa que intenta garantizar que el usuario solo acceda a archivos de una carpeta autorizada (por ejemplo, `/var/www/images/`). La aplicación comprueba explícitamente que la cadena del parámetro empiece por el prefijo esperado (ej. `startsWith("/var/www/images/")`). Sin embargo, si la aplicación valida el inicio de la cadena pero no bloquea la inclusión de secuencias de escape relativas (`../`) dentro del resto del parámetro, la validación se vuelve ineficaz.

Mecanismo de Explotación:

El fallo conceptual reside en que la aplicación satisface su control de seguridad simplemente confirmando la coincidencia del prefijo al inicio de la cadena, sin verificar el destino final resolutorio del parámetro completo. El atacante satisface el filtro manteniendo el prefijo obligatorio al inicio e inmediatamente añade secuencias de escape `../` para "deshacer" esa ruta y retroceder hasta la raíz del sistema de archivos:

Payload enviado: `/var/www/images/../../../../etc/passwd`

Validación del Servidor: ¿Empieza por `/var/www/images/`? → Sí (Pasa la validación)

Resolución del OS: `/var/www/images/../../../../etc/passwd` → `/etc/passwd`

Punto Clave de Seguridad:

Validar únicamente el prefijo de una cadena (coincidencia de texto plano al inicio) no garantiza que la ruta final resolutoria permanezca dentro del directorio autorizado. Un control de entrada seguro debe evaluar el resultado final de la ruta (canonicalización) y no solo la estructura inicial de la cadena de texto recibida.

Impacto en la Tríada CIA:

Confidencialidad (Alto): Es el pilar directamente comprometido. Permite la exfiltración de archivos de sistema críticos como `/etc/passwd`, credenciales de bases de datos en archivos `.env` o configuraciones, claves privadas SSH y código fuente.

Integridad (Bajo): En este reto el vector es exclusivamente de lectura; sin embargo, las credenciales o claves obtenidas podrían facilitar la modificación de datos en ataques posteriores.

Disponibilidad (Medio): La lectura repetida de dispositivos especiales del sistema (`/dev/urandom`, `/dev/zero`) mediante este bypass puede derivar en un consumo desmedido de recursos y causar una Denegación de Servicio (DoS).

Procedimiento de Explotación (Metodología en Burp Suite)

Reconocimiento y Captura de Petición:

- Abrir el navegador integrado de Burp Suite y acceder a la tienda virtual del laboratorio.
- Seleccionar cualquier producto para forzar la carga de su imagen en la aplicación.
- Ir a la pestaña Proxy > HTTP history y ubicar la solicitud encauzada a servir la imagen:

`GET /image?filename=/var/www/images/54.jpg HTTP/1.1`

Envío al Módulo Repeater:

- Hacer clic derecho sobre la petición identificada y seleccionar Send to Repeater (o presionar Ctrl + R).

Prueba de Inyección de Payload (Bypass de Prefijo Obligatorio):

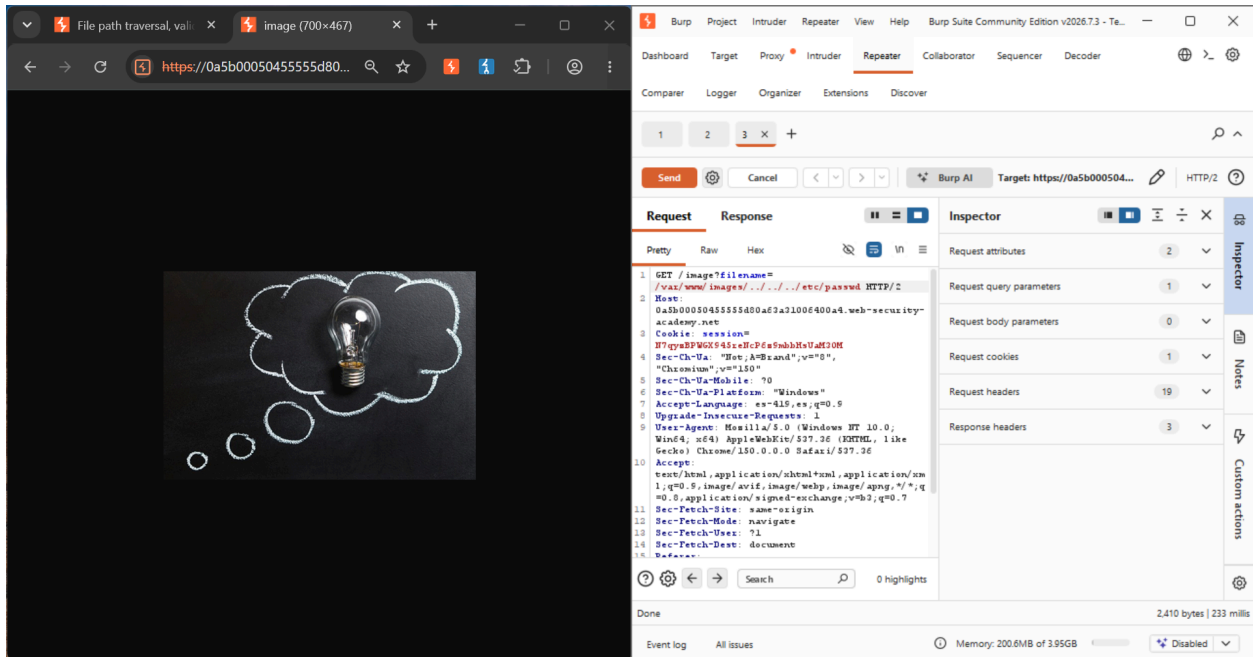
- Conservar intacto el prefijo obligatorio (`/var/www/images/`) al inicio del parámetro `filename`.
- Añadir inmediatamente después la secuencia de escape `../../../../etc/passwd`.
- Estructurar la línea de la solicitud en Repeater de la siguiente manera:
- `GET /image?filename=/var/www/images/../../../../etc/passwd HTTP/1.1`

Exfiltración y Confirmación:

- Hacer clic en el botón Send.
- Verificar que la respuesta devuelta por el servidor sea un código de estado HTTP/2 200 OK.
- Confirmar en el cuerpo de la respuesta (Response Body) la presencia del contenido del

archivo /etc/passwd.

- Refrescar la página principal en el navegador para validar la resolución con el banner "Lab Solved".



Estrategias de Mitigación Recomendadas

Verificación de Ruta Canónica (Canonicalization Check):

En lugar de validar únicamente la cadena de texto inicial mediante `startsWith()`, la aplicación debe resolver la ruta canónica absoluta en el servidor y verificar de forma programática que empiece estrictamente dentro del directorio permitido.

Ejemplo en Java:

```
File file = new File("/var/www/images/", userInput);
```

```
String canonicalPath = file.getCanonicalPath();
```

```
if (!canonicalPath.startsWith("/var/www/images/")) {
```

```
    throw new SecurityException("Intento de Path Traversal detectado: La ruta final escapa del directorio base.");
```

```
}
```

Sanitización Mediante Listas Blancas (Whitelisting):

Si el parámetro solo debe recibir el nombre del archivo (ej. 54.jpg), no se debe requerir ni permitir que el usuario pase rutas completas en la URL. Se debe validar mediante una expresión regular estricta:

```
^[a-zA-Z0-9_-]+\.(jpg|png|gif)$
```

Uso de Claves o Índices Indirectos:

Mapear los archivos en una base de datos o un índice entero (ej. GET /image?id=54) para que la aplicación resuelva la ruta interna en el backend sin depender de entradas del usuario.

Principio de Menor Privilegio:

Ejecutar el proceso del servidor web (www-data) con permisos de lectura mínimos en el sistema operativo, de modo que no tenga privilegios para acceder a /etc/passwd ni a directorios fuera del directorio público.

Conclusiones y Aprendizaje

- Inseguridad de las Validaciones Parciales: Comprobar únicamente el inicio de una cadena (string prefix match) crea una falsa sensación de seguridad, ya que los atacantes pueden cumplir esa condición inicial y añadir caracteres de escape a continuación.
- Cadena de Texto vs. Ruta Resolutiva: Existe una diferencia fundamental entre validar la representación textual de un parámetro recibida en la petición HTTP y validar la ruta real que interpreta el sistema operativo tras resolver los caracteres ../.
- Priorización de Controles Canónicos: Para mitigar ataques de navegación por el sistema de archivos de forma definitiva, los controles de seguridad deben basarse siempre en la resolución canónica de rutas e implementación de listas blancas, descartando las comprobaciones superficiales de texto.